

Introduction

Goal: Costruire un sistema software che comanda un Differential Drive Robot (DDR)

Requirements

Costruire un sistema software che comanda un Differential Drive Robot (DDR) in modo che, partendo dalla posizione iniziale HOME, il robot si sposti lungo il perimetro di una stanza rettangolare vuota. Il DDR si deve fermare quando tornato in HOME. Il DDR è anche sensibile ai dati rilevati da un Radar. Quando il Radar rileva un 'intruso' a una distanza minore di una distanza prefissata ``DMIN``: - se il DDR si sta muovendo , il DDR si ferma fino a che il Radar non rileva più un 'intruso' così vicino - se il DDR è fermo in HOME, il DDR ruota su sè stesso fino a che un 'intruso' così vicino scompare

Requirement analysis

ROOM -> Dopo un analisi con il committente risulta quindi naturale andare a rappresentare la stanza come una griglia, dove ogni cella sarà un'unità robotica. Viene modellata quindi come una griglia rettangolare di base b e altezza h di unità robotiche, $b \geq 1$ unità robotica, $h \geq 1$ unità robotica, in modo che il DDR possa muoversi lungo il perimetro senza uscire dai confini. Nota: Non usiamo i metri ma usiamo unità robotiche (lato del DDR) per semplificare la modellazione e l'implementazione del sistema. HOME_DDR -> è un area di 1 unità robotica in grado di contenere il robot. Ovvero è una cella predefinta all'interno della griglia, in modo che il DDR possa muoversi all'interno di essa senza uscire dai confini.

```
[#
  data class RoomConfiguration(
    val width: Double,
    val height: Double,
    val cellSideinMt: Double
  ) {
    val totalWidthinMt: Double = width * cellSideinMt
    val totalHeightinMt: Double = height * cellSideinMt
    val homePoint: PointInGrid = PointInGrid(0, 0)
  }

  data class PointInGrid(val x: Int, val y: Int)
#]
```

Radar -> Il radar è modellato come un [QActor](#). Esso è un'entità proattiva che controlla i dintorni del DDR rilevando la distanza degli intrusi. Questo attore è quindi una rappresentazione del fatto che il DDR è dotato di un sensore che gli permette di rilevare la presenza di intrusi che si trovano ad una distanza inferiore di DMIN. E' necessario formalizzare questi concetti, il radar comunica con il DDR emettendo un evento.

```
QActor Radar context ctxddr {
  [#
    distanza_intruso = 0; // variabile che rappresenta la distanza dell'intruso più vicino rileva
    // Assumiamo che veda solo davanti a se
  #]
  State s0 initial {
    println("$name | attivo") color green
  }
  Goto emitDistance

  State emitDistance {
    emit infoRadar : distance(distanza_intruso)
    // Evento: distance(distanza_intruso) che rappresenta la distanza dell'intruso rilevato dal r
  }
}
```

DDR -> Dai requisiti si evince che il DDR è proattivo e risponde a comandi di movimento (avanti, indietro, destra, sinistra) e a comandi di stop. Parlando con il committente si arriva alla conclusione che il **DDR** è fornito dal committente stesso ed è inscrivibile in un quadrato. E' possibile comunicare con il DDR con http su 8090 sincrona oppure websocket su 8091 asincrona a 2 vie.



